Verex: Verified Execution

Vision · Move Summit 2026 (internal)

Wolfgang Grieskamp

Aptos Labs

The problem

- Critical software is produced faster than people can review it: AI writes it, agents run it, AI-assisted auditing finds every flaw.
- Assurance today is a chain of trusted components: developer, reviewer, compiler, analysis tools, runtime. Each is a single point of failure.
- Memory-safe languages trust the compiler. Sandboxes see untyped bytes. Verifiers say nothing about bugs in compiler or runtime.

How to run code nobody fully trusts, with guarantees that survive a bug in any one component?

The product: Verex

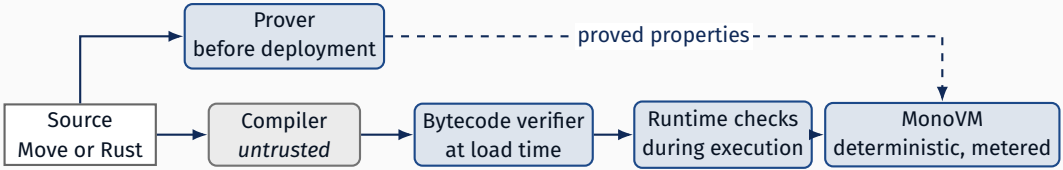
- Teams write ordinary Rust; Verex verifies and runs it.
- Any language with ownership and borrowing; Move, the origin, included.
- Proofs before deployment, independent checks during execution.

What owners get

- Mathematical proofs, not only tests
- Independent redundancy of the checks
- Containment of untrusted code
- Defined failure instead of silent corruption
- Reproducible execution, bit for bit
- Bounded, metered resources

Run code you do not trust, and know what it can and cannot do.

Design principle: no single trusted component



- Every safety invariant (typing, resources, references, memory) is established at least twice, by independent implementations.
- The bytecode verifier trusts nothing from the compiler; the runtime checks trust nothing from the verifier.
- Possible because the type system makes safety checkable on bytecode, and references never alias.

Injected code

- Arrives through the tools the platform does not trust.
- Cannot forge an authorization token, duplicate an asset, or reach a value it holds no reference to.
- Type and resource safety bound the damage.

Two deployment modes

- **MonoVM**: all layers in place.
- **Native**: for real-time loops and numeric kernels; proofs hold, the native compiler is trusted.
- MonoVM stays for simulation, differential testing and shadow execution.

The platform today

Implemented and open source in the Aptos code base; years of production with billions of dollars at stake.

- **Move**: Rust-like types, first-class functions, linear resources, borrow discipline.
- **Move Prover**: contracts and invariants, SMT and Lean, fast enough for CI.
- **Bytecode verifier**: typing, abilities and references at load time.
- **Runtime safety checks**: the same, with separate code; stopped real exploits that passed the verifier.
- **MonoVM**: monomorphized code, flat values, 30–50$\times$ faster; validated by mainnet replay.

Why it fits

- Sequential safe Rust and Move enforce the same aliasing rule.
- So Rust compiles to bytecode whose safety stays checkable; Wasm and native code discard it.

The subset

- Sequential and deterministic, including floating point.
- Recursive types, interior mutability and raw pointers included.
- Everything else is rejected with a diagnostic.

Four bytecode extensions, from borrow-checked MIR

- References inside structs: lifetime parameters, checked by the verifier.
- Recursive types through `Box` and `Rc`, heap objects in MonoVM.
- Interior mutability: native cells with dynamic borrow state.
- Raw pointers with checked semantics.

Unsafe code under a checked memory model

Native Rust: raw-pointer misuse is undefined behavior. Verex: every raw-pointer operation is checked.

- Strict provenance, allocation identities never reused, typed allocations.
- Use after free, double free, out of bounds, invalid reinterpretation: a located abort.
- Raw pointer to reference is a dynamic reborrow, checked like any reference.
- The prover discharges the same conditions; where no check can fail, the VM skips it.
- Purpose: reuse existing Rust crates without rewriting them.

- **Automated**: the Move Prover's methods carry over, since they rest on the shared reference discipline; SMT in CI.
- **Kernel-checked**: the Lean-based verifier proves a contract as a theorem about the program's semantics; absence of undefined behavior included.
- **Specification inference**: abort and postconditions inferred mechanically, invariants and intent with AI. People review specifications instead of writing them.
- **Numeric code**: specified over the reals with error bounds; deterministic floating point makes results reproducible.

Evidence to date

- Prover in CI on the Aptos framework: block prologue, staking, metering, code deployment.
- Runtime checks stopped proof-of-concept exploits that passed the static verifier.
- Reference checker replayed billions of transactions without a false alarm.
- Runtime check cost: about 10% for types and abilities; 26–29% for references, before optimization.
- MonoVM: 30–50$\times$ the current Move VM on recorded blocks.

Comparisons with WebAssembly runtimes and native Rust are in preparation.

Application domains

- **Agent-generated code**: metered, isolated, replayable, with inferred and proved properties.
- **Financial logic**: settlement, clearing, risk limits, pricing; resources model assets, invariants express conservation.
- **Safety-critical and defense**: proved control laws, bounded resources, reproducible numerics.
- **Multi-party computation**: run a partner's or regulator's code without trusting their toolchain.
- **Existing Rust**: plugins and libraries with unsafe internals, hardened without rewriting.

Adoption starts with one component where failure is most expensive, run next to the existing system.

Relation to existing approaches

	Proves	Checks at load	Runtime checks	Deterministic
Verex	✓	✓	✓	✓
Wasm runtimes	×	◐	◐	◐
JVM, CLR	×	✓	◐	×
Rust verifiers (Verus, Kani, …)	✓	×	×	×
SPARK / Ada	✓	×	◐	×
K framework	✓	×	◐	◐
Miri	×	×	◐	×
zkVMs	×	×	×	✓

- zkVMs prove that code ran, not that it is correct; complementary.
- Specialized, not generic: every layer relies on the ownership discipline.

✓ yes ◐ partial, e.g. memory bounds only, or left to the embedder × no

Roadmap

0–4 months	Rust Stage 0 pipeline; deterministic floating point; checked raw pointers in the Lean verifier; MonoVM standalone; benchmarks against Wasm and native; first design partners
4–8 months	Lifetime-annotated bytecode and recursive types; checked raw pointers in the VM; prover for the Rust subset; production reference checker; specification inference as a product
8–14 months	Lifetime-checked verifier for untrusted Rust bytecode; verified library models; compiler correctness for core fragments; ahead-of-time compilation; tool qualification groundwork

Assumes sufficient staffing.

Open problems

- **Sound lifetime checking of adversarial bytecode**: a new verifier algorithm is needed.
- **Ecosystem coverage**: how much real Rust fits the subset.
- **Fidelity for unsafe code**: no normative Rust memory model exists.
- **Performance against native**: JIT or AOT from verified bytecode that keeps the runtime checks.
- **Specification effort**: loop invariants and numeric error bounds remain hardest.
- **Certification**: tool qualification and long-term support.

Questions?

Whitepaper: *Verex: Verified Execution*, draft of Oct 6, 2026